Vector space of bit vectors

Ralf Poeppel
mailto:ralf@poeppel-familie.de

2026-07-19

Abstract

The aim of this article is to document all properties of the vector space of size n of bit vectors $\mathbb{F}_2^n$ conscisly in one place. We include the properties usually only documented for the vector spaces $\mathbb{Q}^n, \mathbb{R}^n, \mathbb{C}^n$, like base, span, dimension, norm, scalar product and inner sum. For the properties norm and scalar product we provide definitions and show that the corresponding axioms are satisfied.

1 Introduction

Bit vectors are frequently used in computer science. They are used for integers, combinatorial algorithms, coding theory, and for logical and arithmetic operations [9, 7]. A set of bit vectors of the same length forms a vector space. Information of all aspects of the vector space of bit vectors are scattered over multiple documents. This article is a collection of the properties of the vector space of bit vectors of the same length.

Bits are based on the finite set of integers of order 2. This set $\{0, 1\}$ with the operations addition and multiplication modulo 2 satisfies the axioms of a field. This finite field is named Galois Field [10, 17][1] $GF(2) = \mathbb{F}_2$. Over this field there is the vector space $\mathbb{F}_2^n$.

The types and functions for the vector space of bit vectors described in this paper are implemented in the Go package `gf2vs` [12]. This article is an enhanced documentation of the Go package.

2 Field $\mathbb{F}_2$

2.1 Supporting Set

Digital computer work with 2 states, mostly electric voltage, on and off. In computer science the two states are named bit and are modeled as the set $\{0, 1\}$. This is the set $\mathbb{Z}_2 \subset \mathbb{N} \subset \mathbb{Z} \subset \mathbb{R}$. This set is equivalent to the set of representatives of the cyclic group $\mathbb{Z}/2\mathbb{Z}$ of order 2 in algebra. In logic we have the boolean values False $F = 0$ and True $T = 1$ [8].

The representatives of the cyclic set $\mathbb{Z}/2\mathbb{Z}$ with operations modulo 2 satisfy the algebraic field axioms. We use the notation $\mathbb{F}_2$ for the field. It is equivalent to the Galois Field $GF(2)$.

2.2 Group Operations

The operations of $\mathbb{F}_2 = \mathbb{Z}/2\mathbb{Z}$ are defined modulo 2; see [3, ch. 2.2.6] and [7].

[1] We cite Wikipedia for reused wordings.

The operations addition and multiplication of the field $\mathbb{F}_2$ satisfy the group axioms [3, 10, 18], both operations are commutative.

Similarly the operations addition and multiplication of the field $\mathbb{R}$ satisfy the group axioms [3], both operations are commutative.

Remark: Please note the different definition of the addition in $\mathbb{F}_2$ and $\mathbb{R}$.

For reference we give here only the operations for $\mathbb{F}_2$.

2.2.1 Addition

The addition is named exclusive disjunction in logic and XOR in computer science [8, 16]. The definition of addition is given in equation (1) obeying $(1 + 1) \bmod 2 = 0$.

$$\begin{aligned} + : \mathbb{F}_2 \times \mathbb{F}_2 \to \mathbb{F}_2, \quad (a, b) \mapsto (a + b) \bmod 2, \\ 0 + 0 = 0, \quad 0 + 1 = 1, \quad 1 + 0 = 1, \quad 1 + 1 = 0. \end{aligned} \tag{1}$$

The group axioms [3, ch. 2.2.8] for the Group $G = \mathbb{F}_2$ and the operation addition are satisfied:

Associativity
$\forall a, b, c \in G : (a + b) + c = a + (b + c)$.

Identity element $e = 0$
$\exists e \in G, \forall a \in G : e + a = a$ and $a + e = a$, $e = 0$, e is unique.

Inverse element $(-a) = a$
$\forall a \in G \;\; \exists (-a) \in G : a + (-a) = e$ and $(-a) + a = e$, e identity element, $(-a) = a$ is unique for each a. The elements of the set $\{0, 1\}$ are their own inverses, see equation (1).

Commutativity
$a + b = b + a$.

So $\mathbb{F}_2$ with the operation addition is an abelian group.

2.2.2 Multiplication

The multiplication is named conjunction in logic and AND in computer science [8, 20]. The multiplication is identically defined as in $\mathbb{Z}$.

$$\begin{aligned} \cdot : \mathbb{F}_2 \times \mathbb{F}_2 \to \mathbb{F}_2, \quad (a, b) \mapsto (a \cdot b) \bmod 2, \\ 0 \cdot 0 = 0, \quad 0 \cdot 1 = 0, \quad 1 \cdot 0 = 0, \quad 1 \cdot 1 = 1. \end{aligned} \tag{2}$$

We may use the notation ab instead of $a \cdot b$, omitting the multiplication sign if there is no ambiguity.

The group axioms for the Group $G = \mathbb{F}_2$ and the operation multiplication are satisfied:

Associativity
$\forall a, b, c \in G : (a \cdot b) \cdot c = a \cdot (b \cdot c)$.

Identity element $e = 1$
$\exists e \in G, \forall a \in G : e \cdot a = a$ and $a \cdot e = a$, $e = 1$, e is unique.

Inverse element a^{-1}
$\forall a \in G, a \neq 0, \;\; \exists a^{-1} \in G : a \cdot a^{-1} = e$ and $a^{-1} \cdot a = e$, e is the identity element; the only invertible element is 1, hence $a^{-1} = 1$ for $a = 1$.

Commutativity
$a \cdot b = b \cdot a$.

So $\mathbb{F}_2$ with the operation multiplication is an abelian group.

2.3 Mappings

In addition to the algebraic group operations we have some mappings, applicable to $\mathbb{F}_2$.

2.3.1 Complement

$$\begin{aligned} \neg : \mathbb{F}_2 \to \mathbb{F}_2, \quad \neg x = 1 + x, \\ \neg 0 = 1, \quad \neg 1 = 0. \end{aligned} \tag{3}$$

2.3.2 Disjunction

In boolean algebra we have the operation disjunction, named OR in computer science [8, 21].

$$\begin{aligned} \vee : \mathbb{F}_2 \times \mathbb{F}_2 \to \mathbb{F}_2, \quad (a, b) \mapsto a \vee b, \\ 0 \vee 0 = 0, \quad 0 \vee 1 = 1, \quad 1 \vee 0 = 1, \quad 1 \vee 1 = 1. \end{aligned} \tag{4}$$

The operation $\vee$ does not satisfy the group axioms; there is no inverse element.

2.3.3 Absolute Value

Absolute value $|\cdot|$ maps an element x of $\mathbb{F}_2$ to $\mathbb{N}$:

$$\begin{aligned} |\cdot| : \mathbb{F}_2 \to \mathbb{N}, \quad x \mapsto y, \\ |0| = 0, \quad |1| = 1. \end{aligned} \tag{5}$$

After application of this mapping, we can use the definition of the addition as in $\mathbb{N}$.

2.4 Field Axioms

The set $\mathbb{F}_2$ with the operations addition and multiplication satisfies the field axioms [3, 7].

K1 $\mathbb{F}_2$ with the addition + is an abelian group.

K2 $\mathbb{F}_2 := \mathbb{F}_2 \setminus \{0\}$ with the multiplication $\cdot$ for every element of $\mathbb{F}_2$ is an abelian group.

K3 distributive property [2, 14] is satisfied $\forall a, b, c \in \mathbb{F}_2$

$$\begin{aligned} a \cdot (b + c) = a \cdot b + a \cdot c, \\ (a + b) \cdot c = a \cdot c + b \cdot c. \end{aligned} \tag{6}$$

Obviously $\mathbb{F}_2$ satisfies the field axioms.

3 Bit Vectors

3.1 Definition

We define a bit vector x of size n as a tuple (x_i) of values $x_i \in \mathbb{F}_2$:

$$x := (x_i) := (x_1, x_2, \ldots, x_n), \quad \forall x_i \in \mathbb{F}_2. \tag{7}$$

And we define two distinguished constant bit vectors:

Zero, 0, zero vector all components are 0, identity element of vector addition equation (11).

Ones, 1, ones vector all components are 1, identity element of vector multiplication.

3.2 Mapping of Integers to Bit Vectors

In computers unsigned integers $a \in \mathbb{N}$ are represented by bit vectors. In this paper we consider unsigned integer $a \geq 0$ only. Regarding the special considerations for representation of integers $a < 0$ as bit vectors see [9].

For mapping of a set $\{a_i \in \mathbb{N}\}$ of unsigned integers we compute first the required size n of the target vectors. Obviously if all $a_i = 0$, we get $n = 0$. Otherwise if the the maximum $\max(a_i) > 0$, we use equation (8).

$$n = \lfloor \log_2(\max(a_i)) \rfloor + 1. \tag{8}$$

Knowing n we can use equation (9) to map $a \in \mathbb{N}, 0 \leq a \leq n$.

$$\varphi : \mathbb{N} \to \mathbb{F}_2^n, \quad x = \left(\lfloor \tfrac{a}{2^0} \rfloor \bmod 2,\ \lfloor \tfrac{a}{2^1} \rfloor \bmod 2,\ \ldots,\ \lfloor \tfrac{a}{2^{n-1}} \rfloor \bmod 2 \right). \tag{9}$$

The inverse mapping φ^{-1} is given by equation (10)

$$\varphi^{-1} : \mathbb{F}_2^n \to \mathbb{N}, \quad a = \textstyle\sum_{i=1}^{n} x_i 2^{i-1}. \tag{10}$$

In the programming language Go, as in most programming languages these mapping are used when parsing integer strings and formatting integers. Internally integers are stored as bit vectors compatible with the hardware word size of the target computer for the programming language. This limits on most hardware architectures the vector size to a power of 2 of the vectors used to represent integers. The Go package `gf2vs` supports vector spaces of arbitrary length, up to 64 bit.

3.3 Operations on Bit Vectors

Equation (11) defines bit-wise operations on the bit vectors x, y, z see [3, ch. 2.4.1] and [9, (1), (2), (3)]. As we define the vector operations bit-wise we inherit the group properties of the field $\mathbb{F}_2$.

$$\left.\begin{array}{lrrcr}
\text{Complement} & \neg : \mathbb{F}_2^n \to \mathbb{F}_2^n, & \neg x = y & \Leftrightarrow & \neg x_i = y_i\ , \\
\text{Addition, XOR} & + : \mathbb{F}_2^n \times \mathbb{F}_2^n \to \mathbb{F}_2^n, & x + y = z & \Leftrightarrow & x_i + y_i = z_i, \\
\text{Multiplication, AND} & \cdot : \mathbb{F}_2^n \times \mathbb{F}_2^n \to \mathbb{F}_2^n, & x \cdot y = z & \Leftrightarrow & x_i \cdot y_i = z_i, \\
\text{Disjunction, OR} & | : \mathbb{F}_2^n \times \mathbb{F}_2^n \to \mathbb{F}_2^n, & x|y = z & \Leftrightarrow & x_i \vee y_i = z_i, \\
\text{Scalar multiplication} & \cdot : \mathbb{F}_2 \times \mathbb{F}_2^n \to \mathbb{F}_2^n, & \lambda \cdot x = y & \Leftrightarrow & \lambda \cdot x_i = y_i, \\
\text{Absolut value} & |\cdot| : \mathbb{F}_2^n \to \mathbb{N}^n, & |x| = y & \Leftrightarrow & |x_i| = y_i, \\
\text{Bit vector norm} & \|\cdot\| : \mathbb{F}_2^n \to \mathbb{N}, & \|x\| & \Leftrightarrow & \textstyle\sum_{i=1}^{n} |x_i|, \\
\text{Bit vector scalar product} & \langle , \rangle : \mathbb{F}_2^n \times \mathbb{F}_2^n \to \mathbb{N}, & \langle x, y \rangle & \Leftrightarrow & \|x \cdot y\|.
\end{array}\right\} i = 1, \ldots, n. \tag{11}$$

We define the mapping $|\cdot|$ for formal mapping of a bit vector from $\mathbb{F}_2^n$ to $\mathbb{N}^n$. Details of the operations bit vector norm and bit vector scalar product are given below.

3.4 Bit Vector Norm

The addition in the field $\mathbb{F}_2$ is modulo 2. Hence each sum in $\mathbb{F}_2$ evaluates to either 0 or 1. In particular, for $x \in \mathbb{F}_2^n$ we have

$$\left(\sum_{i=1}^{n} x_i\right) \bmod 2 = \begin{cases} 1, & |\{i \in \{1,\ldots,n\} : x_i = 1\}| \text{ is odd,} \\ 0, & |\{i \in \{1,\ldots,n\} : x_i = 1\}| \text{ is even.} \end{cases} \tag{12}$$

From this it follows that we cannot directly use the usual norm definitions (as over $\mathbb{R}$) to measure vector length in $\mathbb{F}_2^n$.

In coding theory [5, 19] the *Hamming weight* w_H (number of 1-entries) of $x \in \{0,1\}^n$ is defined as:

$$w_H : \mathbb{F}_2^n \to \mathbb{N}, \quad w_H(x) = |\{i \in \{1,\ldots,n\} : x_i = 1\}|, \tag{13}$$

and the associated *Hamming distance* d_H is:

$$d_H : \mathbb{F}_2^n \times \mathbb{F}_2^n \to \mathbb{N}, \quad d_H(x,y) = w_H(x-y). \tag{14}$$

If we first apply operation absolute value $|\ |$ we map a vector from $\mathbb{F}_2^n \to \mathbb{N}^n$, by embedding $\mathbb{F}_2^n = \{0,1\}^n \subset \mathbb{R}^n$. By this mapping we overcome the limitation of equation (12). On vectors $x = (x_i)$ in $\mathbb{R}^n$ norms are defined and we get:

$$\|x\|_1 = \sum_{i=1}^{n} |x_i| = w_H(x). \tag{15}$$

In $\mathbb{R}^n$ $\|x\|_1$ is named ℓ_1-norm see [4, 13]. Obviously it equals the Hamming weight on bit vectors. This norm is sometimes called absolute-value norm [22]. We use the Hamming weight and define it to be the norm $\|x\|$ of a bit vector.

$$\|x\| = \sum_{i=1}^{n} |x_i|. \tag{16}$$

The value of the norm of a vector of $\mathbb{F}_2^n$ is an element of the set $\{0,1,\ldots,n\} \subset \mathbb{N} \subset \mathbb{Z} \subset \mathbb{R}$.

In the programming languages C the function is named popcount() and in the language Go it is the function `OnesCount(x uint) uint` in package `math/bits`.

To prove that equation (16) defines a norm, it must satisfy the three norm axioms: positive definiteness, absolute homogeneity, and the triangle inequality.

Positive definiteness

Non-negativity $\|x\| \geq 0$.

Since the absolute value of a bit x_i is always greater than or equal to zero $|x_i| \geq 0$ the sum of these absolute values is also never negative. $\|x\| = \sum_{i=1}^{n} |x_i| \geq 0$.

Definiteness $\|x\| = 0 \Leftrightarrow x = \mathbf{0}$.

$\Longrightarrow$: Let $\|x\| = 0$. This means $\sum_{i=1}^{n} |x_i| = 0$. Since the absolute value of a bit is never negative ($|x_i| \geq 0$), this sum can equal 0 only if every single term is 0. Thus $|x_i| = 0$ must hold for all $i \in \{1,2,\ldots,n\}$. It follows a vector with all $x_i = 0$, corresponds exactly to the zero vector $x = \mathbf{0}$.

$\Longleftarrow$ **:** If $x = \mathbf{0}$, then each component $x_i = 0$. Consequently, $|x_i| = 0$ as well, and the sum equals zero: $\|\mathbf{0}\| = \sum_{i=1}^{n} |0| = 0$.

Absolute homogeneity $\|\lambda \cdot x\| = \lambda \cdot \|x\|$.

Factoring a scalar out of the norm results in that factor being multiplied as an absolute value. For any scalar $\lambda \in \mathbb{F}$ and any vector $x \in \mathbb{F}^n$, the following holds: $\|\lambda x\| = \sum_{i=1}^{n} |\lambda x_i|$ Using the multiplicative property of the absolute value $|\lambda x_i| = |\lambda| \cdot |x_i|$, we obtain: $\|\lambda x\| = \sum_{i=1}^{n} (|\lambda| \cdot |x_i|)$ Since the scalar $|\lambda|$ is constant for all terms in the sum, we can factor it out: $\|\lambda x\| = |\lambda| \cdot \sum_{i=1}^{n} |x_i| = |\lambda| \cdot \|x\|$.

Subadditivity / Triangle inequality $\|x + y\| \leq \|x\| + \|y\|$,

The norm of the sum of two vectors is never greater than the sum of the individual norms. For two vectors $x, y \in \mathbb{F}^n$, the following holds: $\|x + y\| = \sum_{i=1}^{n} |x_i + y_i|$. From equation (1) and the definition of the absolut

value we know the absolut value of the sum of two numbers $x_i, y_i \in \mathbb{F}$ is never greater than the sum of their absolute values $|x_i+y_i| \leq |x_i|+|y_i|$. Based on the rules for summation, we can split $\|x+y\| \leq \sum_{i=1}^{n}(|x_i|+|y_i|)$ into two separate sums: $\|x+y\| \leq \sum_{i=1}^{n}|x_i| + \sum_{i=1}^{n}|y_i|$. This corresponds exactly to the definition of the norms of the individual vectors: $\|x+y\| \leq \|x\| + \|y\|$.

Since the bit vector norm satisfies all three axioms, it is a valid norm from a mathematical perspective. □

3.5 Bit Vector Scalar Product

We define the scalar product, dot product [3, 15] or inner product of two vectors as given in equation (17):

$$\langle , \rangle : \mathbb{F}_2^n \times \mathbb{F}_2^n \to \mathbb{R}, \quad \langle x, y\rangle = \sum_{i=1}^{n} |x_i \cdot y_i| = \|x \cdot y\| = w_H(x \cdot y). \tag{17}$$

The obtained value equals the standard scalar product of $x, y \in \{0,1\}^n \subset \mathbb{R}^n$

$$\langle , \rangle : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}, \quad \langle x, y\rangle = \sum_{i}^{n} x_i \cdot y_i. \tag{18}$$

By computation we can easily proof the properties of the scalar product, for $x, y \in \mathbb{F}_2^n$ and $\lambda \in \mathbb{F}_2$:

Bilinearity

Distributivity in the first argument: $\langle x + x', y\rangle = \langle x, y\rangle + \langle x', y\rangle$.

$$\begin{aligned}
\langle x + x', y\rangle &= \textstyle\sum_{i=1}^{n} |(x_i + x_i')y_i| && \text{(definition)},\\
&= \textstyle\sum_{i=1}^{n} |(x_i y_i + x_i' y_i)| && \text{(distributiv law for bits)},\\
&= \textstyle\sum_{i=1}^{n} |x_i y_i| + \sum_{i=1}^{n} |x_i' y_i| && \text{(sum splitting)},\\
&= \langle x, y\rangle + \langle x', y\rangle.
\end{aligned}$$

Distributivity in the second argument: $\langle x, y + y'\rangle = \langle x, y\rangle + \langle x, y'\rangle$.

$$\begin{aligned}
\langle x, y + y'\rangle &= \textstyle\sum_{i=1}^{n} |x_i(y_i + y_i')| && \text{(definition)},\\
&= \textstyle\sum_{i=1}^{n} |(x_i y_i + x_i y_i')| && \text{(distributiv law for bits)},\\
&= \textstyle\sum_{i=1}^{n} |x_i y_i| + \sum_{i=1}^{n} |x_i y_i'| && \text{(sum splitting)},\\
&= \langle x, y\rangle + \langle x, y'\rangle.
\end{aligned}$$

Homogeneity in the first argument: $\langle \lambda x, y\rangle = \lambda\langle x, y\rangle$.

$$\begin{aligned}
\langle \lambda x, y\rangle &= \textstyle\sum_{i=1}^{n} |(\lambda x_i)y_i| && \text{(definition)},\\
&= \textstyle\sum_{i=1}^{n} |\lambda(x_i y_i)| && \text{(associative law of bits)},\\
&= |\lambda| \textstyle\sum_{i=1}^{n} |x_i y_i| && (|\lambda| \text{ independent of summation index}),\\
&= \lambda\langle x, y\rangle && (|\lambda| = \lambda).
\end{aligned}$$

Homogeneity in the second argument: $\langle x, \lambda y\rangle = \lambda\langle x, y\rangle$.

$$\begin{aligned}
\langle x, \lambda y\rangle &= \textstyle\sum_{i=1}^{n} |x_i(\lambda y_i)| && \text{(definition)},\\
&= \textstyle\sum_{i=1}^{n} |\lambda(x_i y_i)| && \text{(commutative and associative laws of bits)},\\
&= |\lambda| \textstyle\sum_{i=1}^{n} |x_i y_i| && (|\lambda| \text{ independent of summation index}),\\
&= \lambda\langle x, y\rangle && (|\lambda| = \lambda).
\end{aligned}$$

Commutativity $\langle x, y\rangle = \langle y, x\rangle$.

$$\begin{aligned}
\langle x, y\rangle &= \textstyle\sum_{i=1}^{n} |x_i y_i)| && \text{(definition)},\\
&= \textstyle\sum_{i=1}^{n} |y_i x_i)| && \text{(commutative law of bits)},\\
&= \langle y, x\rangle.
\end{aligned}$$

Positive definiteness

Non-negativity $\langle x, x\rangle \geq 0$.

Since the absolute value of a product of bits $x_i \cdot x_i$ is always greater than or equal to zero $|x_i \cdot x_i| \geq 0$ the sum of these absolute values is also never negative. $\langle x, x\rangle = \sum_{i=1}^{n} |x_i \cdot x_i| \geq 0$.

Definiteness $\langle x, x\rangle = 0 \Leftrightarrow x = \mathbf{0}$.

$\Longrightarrow$: Let $\langle x, x\rangle = 0$. This means $\sum_{i=1}^{n} |x_i \cdot x_i| = 0$. This sum can equal 0 only if every single term is 0. Thus $x_i \cdot x_i = x_i = 0$ must hold for all $i \in \{1, 2, \ldots, n\}$. It follows a vector with all $x_i = 0$, corresponds exactly to the zero vector $x = \mathbf{0}$.

$\Longleftarrow$ **:** If $x = \mathbf{0}$, then each component $x_i = 0$. Consequently, $|x_i \cdot x_i| = 0$ as well, and the sum equals zero: $\langle \mathbf{0}, \mathbf{0}\rangle = \sum_{i=1}^{n} |0 \cdot 0| = 0$.

Orthogonality $x \perp y : \langle x, y\rangle = 0, \langle x, \neg x\rangle = 0 \Leftrightarrow x \perp \neg x$

$\Longrightarrow$: Let $\langle x, \neg x\rangle = 0$. This means $\sum_{i=1}^{n} |x_i \cdot \neg x_i| = 0$. Since $x_i \cdot \neg x_i = 0$ for all $i \in \{1, \ldots, n\}$. So $x \perp \neg x$.

$\Longleftarrow$ **:** If $x \perp \neg x$, then $x_i \cdot \neg x_i = 0$ for each component. Consequently, $\langle x, \neg x\rangle = 0$ as well.

In coding theory [11] the orthogonal bit vector is called the dual code.

Since the bit vector scalar product, defined here, satisfies all axioms of a scalar product it is a scalar product, bilinear form in mathematical perspective. □

3.6 Identities

This chapter lists the identies of operation on bit vectors of size n. D. E. Knuth [9, (4), ..., (14)] provides most of them.

$\neg\mathbf{0} = \mathbf{1},\ \neg\mathbf{1} = \mathbf{0}$: complement of constants; (19)

$x + y = y + x,\ x \cdot y = y \cdot x,\ x|y = y|x$: commutativity; (20)

$(x + y) + z = x + (y + z),\ (x \cdot y) \cdot z = x \cdot (y \cdot z),\ (x|y)|z = x|(y|z)$: associativity; (21)

$(x + y) \cdot z = (x \cdot z) + (y \cdot z)$: distributivity AND over XOR; (22)

$(x \cdot y)|z = (x|z) \cdot (y|z),\ (x|y) \cdot z = (x \cdot z)|(y \cdot z)$: mutual distributivity AND, OR; (23)

$(x \cdot y) + (x|y) = x + y$; (24)

$(x \cdot y)|x = x,\ (x|y) \cdot x = x$: idempotence; (25)

$x + \mathbf{0} = x,\ x \cdot \mathbf{0} = \mathbf{0},\ x|\mathbf{0} = x$; (26)

$x + \mathbf{1} = \neg x,\ x \cdot \mathbf{1} = x,\ x|\mathbf{1} = \mathbf{1}$; (27)

$x + x = \mathbf{0},\ x \cdot x = x,\ x|x = x$; (28)

$x + (\neg x) = \mathbf{1},\ x \cdot (\neg x) = \mathbf{0},\ x|(\neg x) = \mathbf{1}$; (29)

$\neg(x + y) = (\neg x) + y = x + (\neg y)$; (30)

$\neg(x \cdot y) = (\neg x)|(\neg y),\ \neg(x|y) = (\neg x) \cdot (\neg y)$: De Morgan; (31)

$\|\mathbf{0}\| = 0,\ \|\mathbf{1}\| = n$; (32)

$\langle \mathbf{1}, \mathbf{1}\rangle = n,\ \langle x, \neg x\rangle = 0$; (33)

4 Vector Space $\mathbb{F}_2^n$

and we define the set of all bit vectors of size n see [3, ch 2.4.1] as the vector space $\mathbb{F}_2^n$:

$$\mathbb{F}_2^n := \{x = (x_1, \ldots, x_n) : x_i \in \mathbb{F}_2\}. \tag{34}$$

4.1 Axioms of Vector Space

The set $\mathbb{F}_2^n$ with the binary operation of vector addition + and the binary function of scalar multiplication $\cdot$, as given in equation (11), define a vector space see [3, 24]. We repeat here the axioms of a vector space and leave the proof to the reader.

Associativity of vector addition
$u + (v + w) = (u + v) + w, \ \ \forall u, v, w \in \mathbb{F}_2^n$.

Commutativity of vector addition
$u + v = v + u, \ \ \forall u, v \in \mathbb{F}_2^n$.

Identity element of vector addition
$\exists \mathbf{0} \in \mathbb{F}_2^n : v + \mathbf{0} = v, \ \ \forall v \in \mathbb{F}_2^n$.

Inverse elements of vector addition
$\forall v \in \mathbb{F}_2^n \ \ \exists -v \in \mathbb{F}_2^n : v + (-v) = \mathbf{0}$, and $-v = v$, i.e. each vector is its own additive inverse, see equation (1).

Compatibility of scalar multiplication with field multiplication
$\lambda(\eta v) = (\lambda\eta)v, \ \ \lambda, \eta \in \mathbb{F}_2, \ v \in \mathbb{F}_2^n$.

Identity element of scalar multiplication
$1v = v, \ \ 1 \in \mathbb{F}_2, \ v \in \mathbb{F}_2^n$, where 1 is the multiplicative identity of $\mathbb{F}_2$.

Distributivity of scalar multiplication with respect to vector addition
$\lambda(u + v) = \lambda u + \lambda v, \ \ \lambda \in \mathbb{F}_2, \ u, v \in \mathbb{F}_2^n$.

Distributivity of scalar multiplication with respect to field addition
$(\lambda + \eta)v = \lambda v + \eta v, \ \ \lambda, \eta \in \mathbb{F}_2, \ v \in \mathbb{F}_2^n$.

The axioms of a vector space are satisfied for $\mathbb{F}_2^n$

4.2 Vector Space Base

For the vector space $\mathbb{F}_2^n$ we give here the definition of the base, the norm and the scalar product as implemented in the Go package.

4.2.1 Unit Vector

We define the unit vectors

$$e_i = (0, \ldots 0, 1, 0, \ldots, 0), \quad i = 1, \ldots, n, \tag{35}$$

of the vector space as the vectors where the ith element is 1 and all other elements are 0.

$$e_i = (x_k), \ \ e_i \in \mathbb{F}^n, \ \ x_k \in \mathbb{F}, \ \ x_k = \begin{cases} 1, & k = i, \quad \text{identity element of multiplication,} \\ 0, & k \neq i, \quad \text{identity element of addition.} \end{cases} \tag{36}$$

Please note the e_i are linearly independent.

Please note the norm of the unit vectors

$$\|e_i\|_1 = 1, \quad \forall i \in \{1, \ldots n\}. \tag{37}$$

These are the only vectors with norm 1 in the vector space.

In the vector space $\mathbb{F}_2^n$ each vector x satisfies the equation (38) iff x is a unit vector [1, 6]

$$(x \neq \mathbf{0}) \wedge ((x \cdot (x + e_1))) = \mathbf{0}. \tag{38}$$

The equation (38) is used in the function `x.IsBaseVector()` of `gf2vs.go`. The algorithm is the fastest test to verify a vector is a unit vector of $\mathbb{F}_2^n$ for arbitrary n.

We observe the identity elements are identical for the fields and the unit vectors are identical for all vector spaces over a field K.

4.2.2 Index

We call $i = 1, \ldots, n$ the index of the unit vector e_i in the base. In the software package `gf2vs.go` we use the Go library function `math/bits.Len(x uint)` for computing the index of a given unit vector. The function `math/bits.Len(x)` computes the number of bits required to represent an unsigned integer. This is an optimized implementation of equation (8).

4.2.3 Generating System

We define the subset $\mathbb{E} := \{e_i\}, \mathbb{E} \subset \mathbb{F}_2^n$, of unit vectors e_i. The subset $\mathbb{E}$ forms a generating system. Each vector v of $\mathbb{F}_2^n$ is a linear combination of scalars a_i and the e_i:

$$v = \sum_{i=1}^{n} a_i e_i, \quad a_i \in \mathbb{F}_2, \quad e_i \in \mathbb{E}, \quad \forall v \in \mathbb{F}_2^n. \tag{39}$$

For the vector space $\mathbb{F}_2^n$ the addition is modulo 2.

Equation (39) is used equally for each vector space on any field K using the operation addition as defined for the field K and the 1 the identity element of the operation multiplication.

Thus the subset $\mathbb{E}$ spans $\mathbb{F}_2^n$. In this vector space it is one spanning set, and the decomposition of a vector v into a linear combination of unit vectors e_i is unique.

4.2.4 Base

The subset $\mathbb{E}$ is one base of the vector space $\mathbb{F}_2^n$. As it is a base of every vector space over a field K.

For the vector space $\mathbb{F}_2^n$ the sum of the base vectors is $\mathbf{1}$ the ones vector..

$$\sum_{i=1}^{n} e_i = \mathbf{1} \tag{40}$$

4.2.5 Orthonormal Base

With the properties of the norm and the scalar product it follows $\mathbb{E}$ is an orthonormal base, and this is the only orthonormal base of $\mathbb{F}_2^n$, as the unit vectors are the only vectors in $\mathbb{F}_2^n$ with a length of 1 in the vector space, see equation (37).

4.3 Sets of Bit Vectors

4.4 Subspaces

4.4.1 Definition

As for other vector spaces, we can define subspaces. A subset U of $\mathbb{F}_2^n$ is a subspace iff U satisfies the three conditions [2, Theorem 1.34].

additive identity
$\mathbf{0} \in U$.

closed under addition
$u, w \in U \Rightarrow u + w \in U$.

closed under scalar multiplication
$a \in \mathbb{F}, u \in U \Rightarrow au \in U$.

For $\mathbb{F}_2^n$ every subset of $\mathbb{E}$ spans a subspace. The conditions can be easily verified.

4.4.2 Direct Sum of Subspaces

Every subspace U of $\mathbb{F}_2^n$ is part of a direct sum equal to V as described by [2, Theorem 2.33]. As per definition $\mathbb{F}_2^n$ is a finite-dimensional vector space. Let U be a subspace of $\mathbb{F}_2^n$. Then there exists a subspace W of $\mathbb{F}_2^n$ such that $V = U + W$.

Let B_V be the canonical base of V so $B_V \subseteq \mathbb{E}$ then $B_W = \mathbb{E} \setminus B$ is a base and we have $B_V \cup B_W = \mathbb{E}$. Let W be the subspace spanned by B_W, then $V \cap W = \{\mathbf{0}\}$ and $V + W = \mathbb{F}_2^n$. For the proof see [2].

Suppose we have two disjunctive sets of vectors $V, U, V \cap U = \emptyset$ of vector space $\mathbb{F}_2^n$, each with corresponding bases B_V, B_U. The bases do not necessarily have to be disjoint.

If the span $V = B_V$ of one subset V, then V is a subspace of $\mathbb{F}_2^n$. In this situation we can create a subspace W out of U so that the direct sum $U + W = \mathbb{F}_2^n$. This is an application of the Steinitz exchange lemma [3, 23].

Assume without loss of generality $B_V = \{e_1, ..., e_k\}$ and $U = \{u_1, ...u_l\}$. We compute the base B_W of W with equation (41)

$$B_W = \mathbb{E} \setminus B_V \tag{41}$$

With the base B_W we can restrict the set of vectors U to a set of vectors $W = \{w_1, ..., w_l\}$ making W a subspace with base B_W simply by filtering the bit vectors of U with a bit mask m

$$m = \sum_{i=1}^{l} e_i, \quad e_i \in B_w, \tag{42}$$

using equation (43) we get

$$w_i = u_i \cdot m, \quad \forall u_i \in U. \tag{43}$$

Acknowledgements

A sincere thank-you goes to my prior fellow student Laura Rubin for the content review and helpful comments, as well as to my wife for her tireless support.

References

[1] Sean Eron Anderson. *Bit Twiddling Hacks*. 2011. URL: `https://graphics.stanford.edu/~seander/bithacks.html` (visited on Mar. 24, 2026).

[2] Sheldon Jay Axler. *Linear Algebra Done Right*. 4th ed. Cham: Springer Nature; 2024. ISBN: 9783031410260. URL: `https://linear.axler.net/LADR4e.pdf` (visited on Feb. 3, 2026).

[3] Gerd Fischer and Boris Springborn. *Lineare Algebra: Eine Einführung für Studienanfänger*. de. 19th ed. Wiesbaden, Germany: Springer Spektrum, 2020.

[4] Otto Forster. *Analysis / Otto Forster ; 1: Differential- und Integralrechnung einer Veränderlichen*. Wiesbaden: Springer Spektrum; 2016. ISBN: 9783658115449.

[5] R. W. Hamming. “Error detecting and error correcting codes”. In: *Bell System Technical Journal* 29.2 (1950), pp. 147–160. URL: https://dn710109.ca.archive.org/0/items/bstj29-2-147/bstj29-2-147.pdf (visited on Feb. 3, 2026).

[6] Jr . Henry S. Warren. *Hacker's Delight*. 2013. URL: https://github.com/jyfc/ebook/blob/master/02_algorithm/Hacker's%20Delight%202nd%20Edition.pdf (visited on Mar. 24, 2026).

[7] Raymond Hill. *A first course in coding theory*. Oxford: Clarendon Press; 2004. ISBN: 0198538030.

[8] Donald E. Knuth. *The Art of Computer Programming, Vol 4, Fasc 0. Introduction to Combinatorical Algorithms and Boolean Functions*. Vol. 4 Fascicle 0. Addison-Wesley, 2008.

[9] Donald E. Knuth. *The Art of Computer Programming, Vol 4, Fasc 1. Bitwise Tricks and Techniques*. Vol. 4 Fascicle 1. Addison-Wesley, 2008.

[10] Rudolf Lidl and Harald Niederreiter. *Finite fields*. Vol. 20. Encyclopedia of mathematics and its applications, volume 20. Cambridge: Cambridge University Press; 2000. ISBN: 9780511525926. DOI: 10.1017/CBO9780511525926.

[11] Jacobus H. Lint. *Introduction to Coding Theory and Algebraic Geometry*. Vol. 12. Oberwolfach seminars, 12. Basel: Birkhäuser Basel; 1988. ISBN: 9783034892865. DOI: 10.1007/978-3-0348-9286-5.

[12] Ralf Poeppel. *Go package documentation gf2vs*. https://pkg.go.dev/github.com/rpoe/gf2vs. [Online; accessed 10-January-2026]. Jan. 6, 2026.

[13] Eric W. Weisstein. *L^1 − Norm From MathWorld–A Wolfram Resource*. https://mathworld.wolfram.com/L1-Norm.html. [Online; accessed 09-October-2025]. July 27, 2025.

[14] Wikipedia contributors. *Distributive property — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Distributive_property&oldid=1329322251. [Online; accessed 28-January-2026]. 2025.

[15] Wikipedia contributors. *Dot product — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Dot_product&oldid=1328631745. [Online; accessed 2-February-2026]. 2025.

[16] Wikipedia contributors. *Exclusive or — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Exclusive_or&oldid=1316886803. [Online; accessed 12-January-2026]. 2025.

[17] Wikipedia contributors. *Finite field — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Finite_field&oldid=1330855394. [Online; accessed 12-January-2026]. 2026.

[18] Wikipedia contributors. *Group (mathematics) — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Group_mathematics&oldid=1330839314. [Online; accessed 12-January-2026]. 2026.

[19] Wikipedia contributors. *Hamming weight — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Hamming_weight&oldid=1306107874. [Online; accessed 13-January-2026]. 2025.

[20] Wikipedia contributors. *Logical conjunction — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Logical_conjunction&oldid=1324909528. [Online; accessed 12-January-2026]. 2025.

[21] Wikipedia contributors. *Logical disjunction — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Logical_disjunction&oldid=1317551960. [Online; accessed 12-January-2026]. 2025.

[22] Wikipedia contributors. *Norm (mathematics) — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Norm_mathematics&oldid=1326013131. [Online; accessed 14-January-2026]. 2025.

[23] Wikipedia contributors. *Steinitz exchange lemma — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Steinitz_exchange_lemma&oldid=1336271854. [Online; accessed 21-April-2026]. 2026.

[24] Wikipedia contributors. *Vector space — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Vector_space&oldid=1326882436. [Online; accessed 12-January-2026]. 2025.